

```

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

# Load dataset
df = pd.read_csv("617ec7a0-b7f1-423e-b810-23f59803ffb6.csv")

print("="*50)
print("LOGISTIC REGRESSION - AIRLINE SATISFACTION")
print("="*50)
print(f"Dataset: {df.shape[0]} rows, {df.shape[1]} columns")

# Handle missing values
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].mean())

# Feature Engineering - Service Quality Average
service_cols = ['Seat comfort', 'Inflight wifi service', 'Inflight entertainment',
                'Online support', 'Ease of Online booking', 'On-board service',
                'Leg room service', 'Baggage handling', 'Checkin service',
                'Cleanliness', 'Online boarding']
df['Service_Quality_Avg'] = df[service_cols].mean(axis=1)

# Feature Engineering - Total Delay
df['Total Delay'] = df['Departure Delay in Minutes'] + df['Arrival Delay in Minutes']
df['Had_Delay'] = (df['Total Delay'] > 0).astype(int)

# Encode categorical variables
categorical_cols = ['Customer Type', 'Type of Travel', 'Class']
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le
    print(f"{col}: {dict(zip(le.classes_, le.transform(le.classes_)))}")

# Encode target variable
target_encoder = LabelEncoder()
df['satisfaction_encoded'] = target_encoder.fit_transform(df['satisfaction'])
print(f"\nsatisfaction: satisfied={target_encoder.transform(['satisfied'])[0]}, not_satisfied={target_encoder.transform(['not_satisfied'])[0]}")

# Define features
features = ['Customer Type', 'Age', 'Type of Travel', 'Class', 'Flight Distance',
            'Seat comfort', 'Departure/Arrival time convenient', 'Food and drink',
            'Gate location', 'Inflight wifi service', 'Inflight entertainment',
            'Service_Quality_Avg', 'Total Delay', 'Had_Delay']

```

```

        'Online support', 'Ease of Online booking', 'On-board service',
        'Leg room service', 'Baggage handling', 'Checkin service',
        'Cleanliness', 'Online boarding', 'Total Delay', 'Had_Delay',
        'Service_Quality_Avg']

X = df[features]
y = df['satisfaction_encoded']

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train Logistic Regression
log_reg = LogisticRegression(C=1.0, solver='lbfgs', max_iter=1000, class_weight
log_reg.fit(X_train_scaled, y_train)

# Predictions
y_pred = log_reg.predict(X_test_scaled)
y_pred_proba = log_reg.predict_proba(X_test_scaled)[: , 1]

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_proba)

print("\n" + "="*50)
print("MODEL PERFORMANCE METRICS")
print("="*50)
print(f"Accuracy:  {accuracy:.4f} ({accuracy*100:.2f}%)"
print(f"Precision: {precision:.4f} ({precision*100:.2f}%)"
print(f"Recall:    {recall:.4f} ({recall*100:.2f}%)"
print(f"F1-Score:   {f1:.4f} ({f1*100:.2f}%)"
print(f"ROC-AUC:    {roc_auc:.4f} ({roc_auc*100:.2f}%)"
print("="*50)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(f"
        Predicted")
print(f"
        Dissatisfied   Satisfied")
print(f"Actual")
print(f"Dissatisfied {cm[0][0]:>10} {cm[0][1]:>9}")
print(f"Satisfied   {cm[1][0]:>10} {cm[1][1]:>9}")
print("\nInterpretation:")
print(f"True Negatives (Correctly predicted Dissatisfied): {cm[0][0]}")

```

```

print(f"False Positives (Dissatisfied predicted as Satisfied): {cm[0][1]}")
print(f"False Negatives (Satisfied predicted as Dissatisfied): {cm[1][0]}")
print(f"True Positives (Correctly predicted Satisfied):      {cm[1][1]}")

# Visualize Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Dissatisfied', 'Satisfied'],
            yticklabels=['Dissatisfied', 'Satisfied'])
plt.title('Confusion Matrix - Logistic Regression')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

# Classification Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Dissatisfied', 'Satisfied']))

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'Logistic Regression (AUC = {roc_auc:.3f})', linewidth=2)
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier', linewidth=1)
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Airline Satisfaction Prediction')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# Feature Importance
coefficients = log_reg.coef_[0]
coef_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': coefficients,
    'Odds_Ratio': np.exp(coefficients)
})
coef_df['Abs_Coefficient'] = np.abs(coef_df['Coefficient'])
coef_df = coef_df.sort_values('Abs_Coefficient', ascending=False)

print("\n" + "="*50)
print("TOP 10 FEATURES INFLUENCING SATISFACTION")
print("="*50)
print("\nPositive coefficients increase satisfaction, negative decrease satisfaction")
print(coef_df[['Feature', 'Coefficient', 'Odds_Ratio']].head(10).to_string(index=False))

# Visualize top coefficients
plt.figure(figsize=(10, 8))
top_features = coef_df.head(15)
colors = ['green' if x > 0 else 'red' for x in top_features['Coefficient']]
plt.barh(top_features['Feature'], top_features['Coefficient'], color=colors)

```

```

plt.xlabel('Coefficient Value')
plt.title('Top 15 Features Influencing Satisfaction')
plt.axvline(x=0, color='black', linestyle='--', alpha=0.5)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

print("\n" + "="*50)
print("SUMMARY")
print("="*50)
print(f"Dataset: {df.shape[0]} rows, {df.shape[1]} columns")
print(f"Features used: {len(features)}")
print(f"Model Accuracy: {accuracy:.4f} ({accuracy*100:.1f}%)")
print(f"ROC-AUC: {roc_auc:.4f}")

print("\nTop 3 Positive Drivers (Increase Satisfaction):")
for i, (_, row) in enumerate(coef_df[coef_df['Coefficient'] > 0].head(3).iterrows()):
    print(f" {i}. {row['Feature']}: {row['Coefficient']:.4f}")

print("\nTop 3 Negative Drivers (Decrease Satisfaction):")
for i, (_, row) in enumerate(coef_df[coef_df['Coefficient'] < 0].head(3).iterrows()):
    print(f" {i}. {row['Feature']}: {row['Coefficient']:.4f}")

```


=====

LOGISTIC REGRESSION - AIRLINE SATISFACTION

=====

Dataset: 129880 rows, 22 columns
Customer Type: {'Loyal Customer': np.int64(0), 'disloyal Customer': np.int64(1)}
Type of Travel: {'Business travel': np.int64(0), 'Personal Travel': np.int64(1)}
Class: {'Business': np.int64(0), 'Eco': np.int64(1), 'Eco Plus': np.int64(2)}

satisfaction: satisfied=1, dissatisfied=0

=====

MODEL PERFORMANCE METRICS

=====

Accuracy: 0.8319 (83.19%)
Precision: 0.8635 (86.35%)
Recall: 0.8230 (82.30%)
F1-Score: 0.8428 (84.28%)
ROC-AUC: 0.9032 (90.32%)

=====

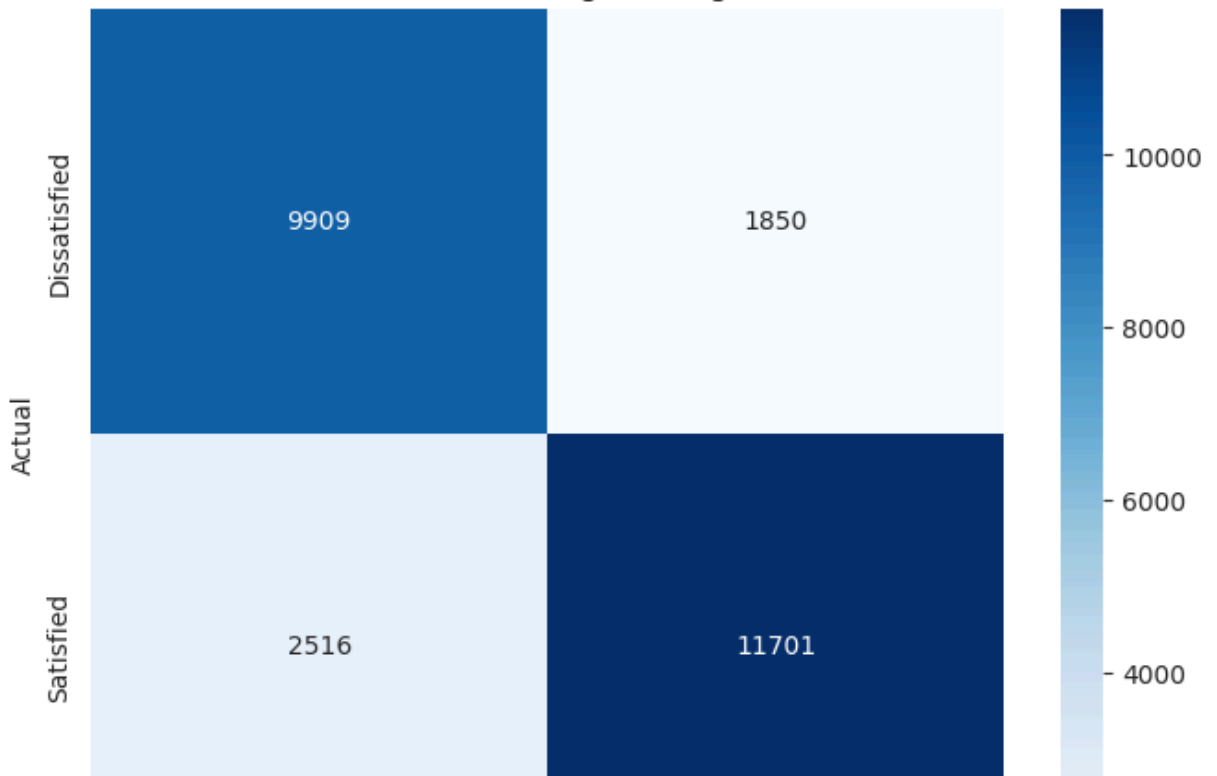
Confusion Matrix:

	Predicted	
	Dissatisfied	Satisfied
Actual		
Dissatisfied	9909	1850
Satisfied	2516	11701

Interpretation:

True Negatives (Correctly predicted Dissatisfied): 9909
False Positives (Dissatisfied predicted as Satisfied): 1850
False Negatives (Satisfied predicted as Dissatisfied): 2516
True Positives (Correctly predicted Satisfied): 11701

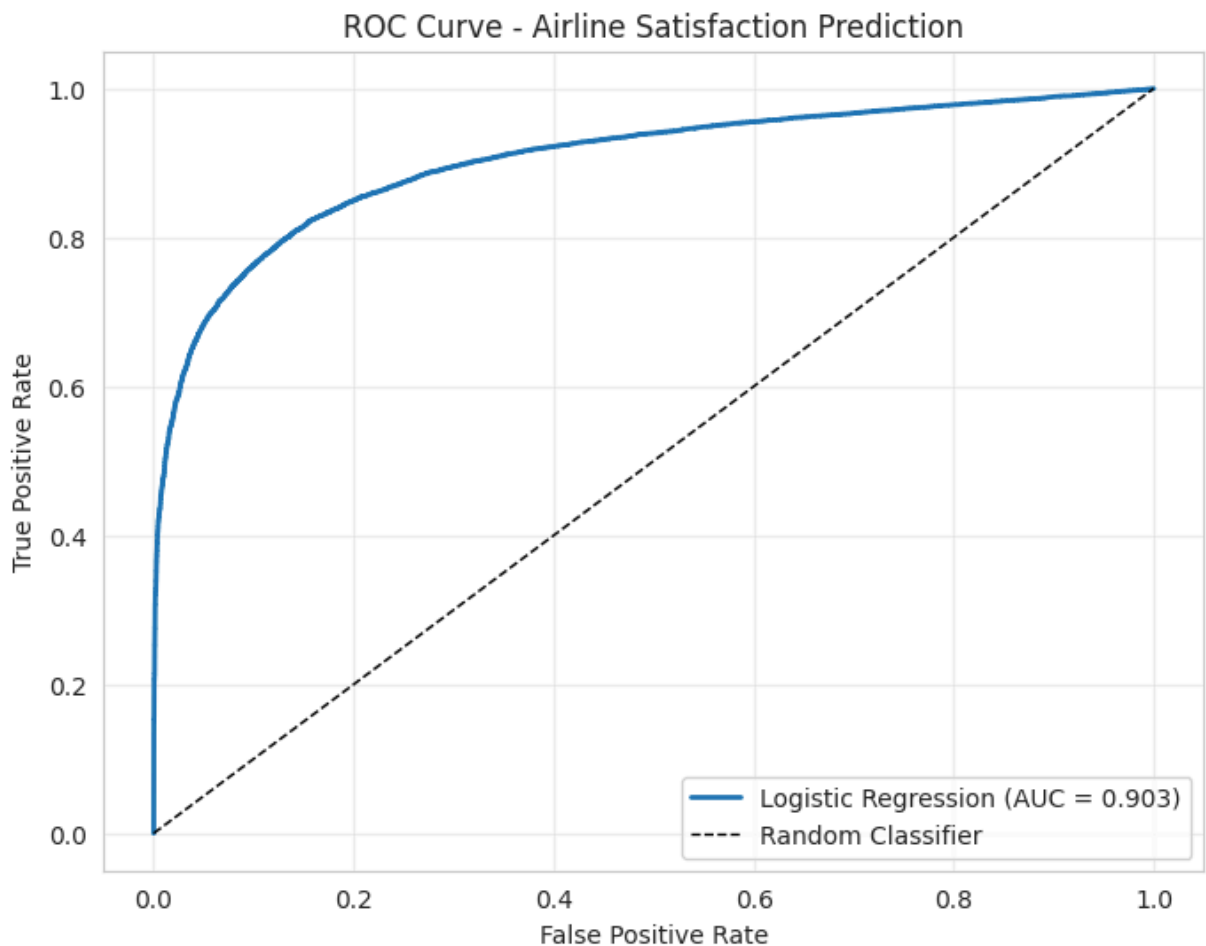
Confusion Matrix - Logistic Regression





Classification Report:

	precision	recall	f1-score	support
Dissatisfied	0.80	0.84	0.82	11759
Satisfied	0.86	0.82	0.84	14217
accuracy			0.83	25976
macro avg	0.83	0.83	0.83	25976
weighted avg	0.83	0.83	0.83	25976



TOP 10 FEATURES INFLUENCING SATISFACTION

Positive coefficients increase satisfaction, negative decrease satisfaction

Feature	Coefficient	Odds_Ratio
Inflight entertainment	0.947759	2.579922
Customer Type	-0.763088	0.466224
Type of Travel	-0.429050	0.651127
Service_Quality_Avg	0.402527	1.495600
On-board service	0.337050	1.400809
Departure/Arrival time convenient	-0.332072	0.717436